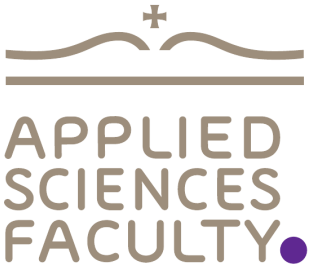
C++ — лекція 18-2. Коди помилок із C++11 (DRAFT!)





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

- Не всі помилки заслуговують на виключення.
  - Приклад-1: в мережевому коді постійно стаються помилки, які є частиною нормальної роботи програми. Наприклад, оскільки мережа принципово ненадійна, варто повідомляти про помилку під'єднання лише після кількох спроб.
  - Приклад-2: асинхронний та багатопоточний код не особливо дружить із виключеннями[1].
  - Приклад-3: виключення прийнятні не всюди, наприклад вони рідко використовуються у вбудованих системах.

[1] Їх використання можливе, але утруднене, і часто втрачаються їх переваги.

Системозалежність

- Помилки є частиною як системних API, так і сторонніх бібліотеки. При чому, вони між собою ніяк не пов'язані.
  - ◇ POSIX стандарт використовує глобальну (TLS) змінну errno. Кожна помилка має свій код.
  - ◇◇ Однак, наприклад, POSIX-варіант getaddrinfo() використовує свої коди помилок[2].
  - ◇ Windows використовує errno лише в межах стандартної бібліотеки. Для WinAPI основним є щось типу GetLastError(), але є варіації.
  - ◇ Сторонні бібліотеки взагалі роблять котра що хоче.
- Все це різноманіття потрібно впорядкувати, без надмірних витрат і у підтримуваний спосіб[3].
  - ◇ Один із наслідків підтримуваності – рішення має включати можливість розширення користувачами.
- На високому рівні зручно мати уніфіковані коди. Однак, трансляція помилок може губити важливу для відлагодження та діагностики інформацію.
  - ◇ Тому важливо мати можливість зберегти вихідний код помилки.

[2] Див. `https://man7.org/linux/man-pages/man3/getaddrinfo.3.html`.

[3] Наприклад, єдиний реєстр всіх кодів помилок – не варіант.

- Враховуючи описані вище потреби, для обробки помилок в C++11 додано дві схожих, але відмінних сутності:
  - Клас **error_code** призначений зберігати вихідний (системний) код помилки.
  - Клас **error_condition** – не залежний від платформи код помилки.
- Обидва цих об'єкти містять вказівник на об'єкт типу **std::error_category** або його нащадка. Цей об'єкт описує, власне, категорію помилки.
  - Приклад категорій з стандартної бібліотеки: std::generic_category, std::system_category, std::iostream_category, future_category().
- Функції, яка може завершитися із помилкою, зазвичай повертатиме **error_code**, але для подальшого аналізу, за можливості, потрібно користуватися error_condition – платформонезалежним варіантом.
- Для взаємодії із виключеннями, до стандарту додано клас виключень **std::system_error**[4], нащадок std::runtime_error, який, окрім методу what(), має метод code(), що повертає код помилки (std::error_code), пов'язаний із помилкою, через яку було кинуто виключення.

[4] З <system_error>.

Перевірка помилки

- Для того, щоб цю машинерію можна було використовувати ефективно, потрібен механізм перетворення між error_code і error_condition.
- Реалізований він так:
  - Різні об'єкти error_code порівнюються оператором порівняння на точне співпадіння.
  - Те ж стосується двох error_condition.
  - Порівняння error_condition і error_code відбувається по іншому – перевіряється їх **еквівалентність**.
- В C++11 деякі коди помилок, (future_errc::broken_promise), мали, помилково, нульові значення – що відповідає загальноприйнятому позначенню "немає помилки". В C++14 це виправили.

Приклад

- Функція, що не кидає виключення:

```
std::error_code ec;
fs::copy_file("no-such-file.txt", "file.txt", ec);
std::cout << "error_code::value = " << ec.value() << std::endl;
std::cout << "error_code::message = " << ec.message() << std::endl;
std::cout << "error_code::category = " << ec.category().name() << std:
if( ec == std::errc::no_such_file_or_directory ){
  std::cout << "Error condition: std::errc::no_such_file_or_directory :
    << static_cast<int>(std::errc::no_such_file_or_directory) << std::e
}
```

- Варіант із виключенням:
- Функція, що не кидає виключення:

```
try{
  fs::copy_file("no-such-file.txt", "file.txt");
} catch (std::system_error& ex) {
  auto ec = ex.code();
            .....
}
```

Стандартні коди помилок

- **std::errc** – class enum, що містить коди типових системних помилок для error_condition.
  - ◇ Наприклад: file_exists (код EEXIST з POSIX), no_such_file_or_directory (ENOENT), not_enough_memory (ENOMEM), protocol_not_supported (EPROTONOSUPPORT) тощо.
  - ◇ Повний список та деякі подробиці: `https://en.cppreference.com/w/cpp/error/errc`.
- **std::future_errc** – error_code (не error_condition!) для помилок std::future і компанії.
- **std::io_errc** – error_code (не error_condition!) для виключення std::ios_base::failure. Стандарт визначає лише один код, stream, але реалізації можуть додавати свої[5].

[5] На момент написання, березень 2022, GCC, Clang, MSVC не визначають інших

Клас error_code

- **class** error_code {
  public:
  error_code() noexcept;
  error_code(**int** ec, **const** error_category& ecat) noexcept;
  template< **class** ErrorCodeEnum >
  error_code(ErrorCodeEnum e) noexcept;
  const error_category& category() **const** noexcept;
  int value() **const** noexcept;
  string message() **const**;
  explicit operator bool() **const** noexcept;
  error_condition default_error_condition() **const** noexcept;
  ...
  };
- Методи менш-більш очевидні. Додаткового обговорення вартують хіба конструктори.
- error_code(int ec, const error_category& ecat) зрозумілий – йому потрібно передати код помилки та її категорію.
  - ◇ Коди помилок унікальні лише в межах категорії.
- error_code(ErrorCodeEnum e) цікавіший – завдяки ADL, вона вибирає ту категорію, що відповідає переданому перерахуванню.
  - ◇ Еквівалентний make_error_code(e). Подробиці – далі, це частина механізму кастомізації.
- default_error_condition() еквівалентний до category().default_error_condition(value()).

- ```
  class error_condition {
  public:
          error_condition() noexcept;
          error_condition( int val, const error_category& cat ) noexcept;
          template< class ErrorConditionEnum >
                  error_condition( ErrorConditionEnum e ) noexcept
          const error_category& category() const noexcept;
          int value() const noexcept;
          string message() const;
          explicit operator bool() const noexcept;
  ...
  };
  ```

- По аналогії із error_code, шаблонний конструктор еквівалентний до make_error_condition(), яка знаходиться за допомогою ADL.
- Інші методи – див. `https://en.cppreference.com/w/cpp/error/error_condition`
  ```

- Оскільки ця презентація тривалий час залишається не завершеною – на жаль, писати її виявилося дуже нудним, ось важливі посилання:
- http://blog.think-async.com/2010/04/system-error-support-in-c0x-part-1.html – огляд, від автора boost::asio.
  - http://blog.think-async.com/2010/04/system-error-support-in-c0x-part-2.html
  - http://blog.think-async.com/2010/04/system-error-support-in-c0x-part-3.html – "Enumerators as class constants",
  - http://blog.think-async.com/2010/04/system-error-support-in-c0x-part-4.html – "Creating your own error codes",
  - http://blog.think-async.com/2010/04/system-error-support-in-c0x-part-5.html – "Creating your own error conditions"
- https://akrzemi1.wordpress.com/2017/07/12/your-own-error-code/ – альтернативна серія статей.
  - https://akrzemi1.wordpress.com/2017/08/12/your-own-error-condition/
  - https://akrzemi1.wordpress.com/2017/09/04/using-error-codes-effectively/
  - https://akrzemi1.wordpress.com/examples/error_code-example-2/
- https://en.cppreference.com/w/cpp/error/error_category
  - https://en.cppreference.com/w/cpp/error/error_code
  - https://en.cppreference.com/w/cpp/error/errno_macros

- Boost::Outcome – `https://www.boost.org/doc/libs/develop/libs/outcome/doc/html/index.html`,
  - `https://www.boost.org/doc/libs/develop/libs/outcome/doc/html/motivation/plug_error_code.html` – "Plugging a library into std::error_code".
  -

